أكاديمية طويق
Tuwaiq Academy

Distributed Computing Principles

Distributed Computing Principles

Introduction to Distributed Computing

What is Distributed Computing?

- Computing paradigm where components on networked computers communicate and coordinate to achieve a common goal.

In everyday terms:

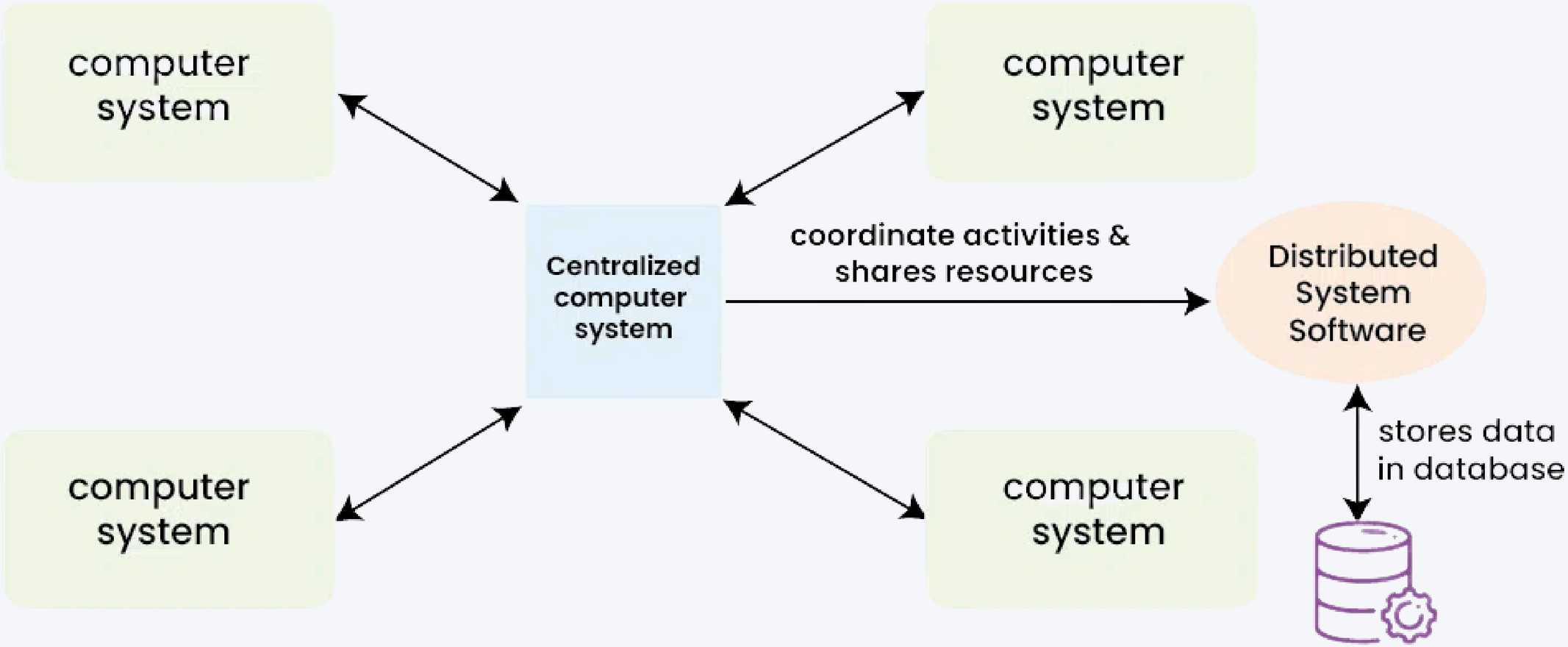
Solving big problems by dividing them among multiple computers that work together

DISCUSSION

Think about it: What systems do you use daily that rely on distributed computing? (Social media, email, streaming services, online banking...)

Distributed Computing Principles

Introduction to Distributed Computing



Distributed Computing Principles

Introduction to Distributed Computing

What is Distributed Computing?

- Computing paradigm where components on networked computers communicate and coordinate to achieve a common goal.

In everyday terms:

Solving big problems by dividing them among multiple computers that work together

DISCUSSION

Think about it: What systems do you use daily that rely on distributed computing? (Social media, email, streaming services, online banking...)

Distributed Computing Principles

The Evolution of Distributed Computing

Era	Paradigm	Example Technologies
1970s-80s	Early networks	ARPANET, DNS
1990s	Client-server	Web servers, databases
2000s	Grid computing	Scientific computing
2010s	Big data frameworks	Hadoop, Spark
Current	Cloud-native	Kubernetes, serverless

Distributed Computing Principles

Why Distributed Computing Exists

```python
# Try running this on your laptop with 1 billion records!
import pandas as pd
import numpy as np

# This would require ~30GB RAM - beyond most laptops
df = pd.DataFrame(np.random.randn(1_000_000_000, 4),
                  columns=['A', 'B', 'C', 'D'])
result = df.groupby(df['A'] > 0).mean()
```

Distributed Computing Principles

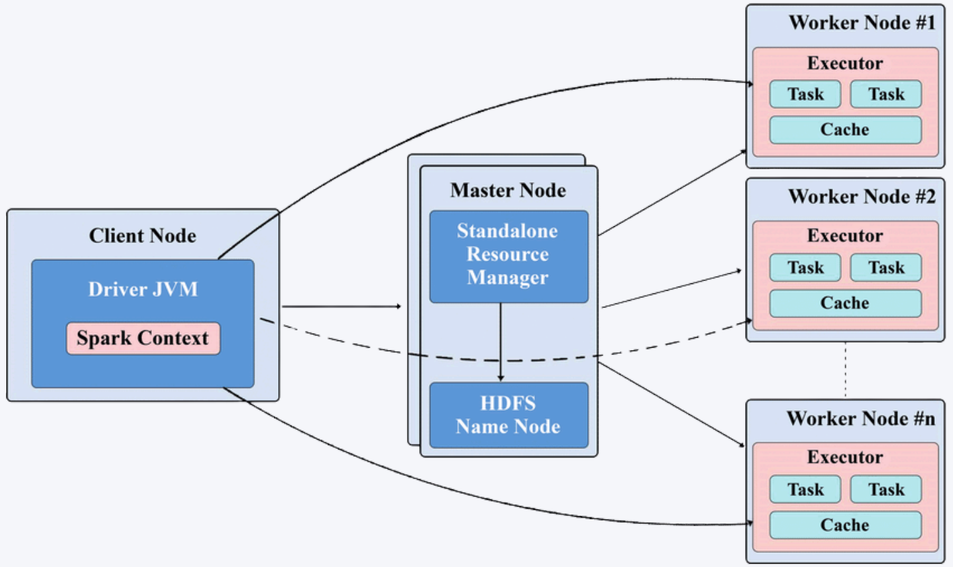
Fundamental Concepts in Distributed Systems

أكاديمية طويق
Tuwaiq Academy

Core Building Blocks

Nodes & Clusters:

- Node: Individual computer/server in the network
- Cluster: Collection of nodes working together
- Example: Spark cluster with 1 master node, 20 worker nodes

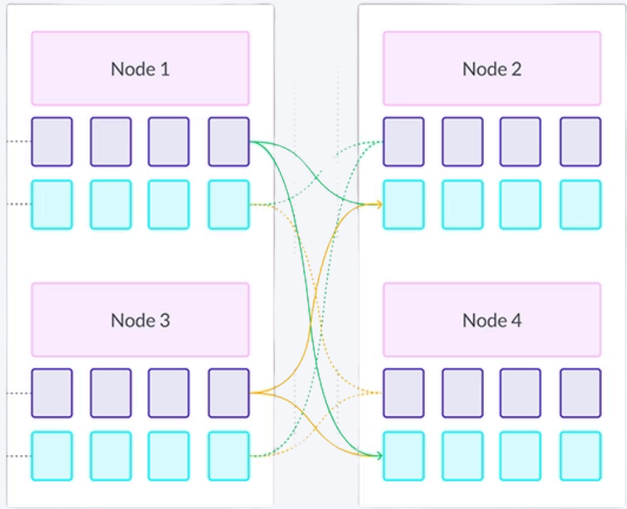


Partitioning & Sharding:

- Dividing data across multiple nodes
- Strategies: range-based, hash-based, directory-based

Replication

- Creating redundant copies of data
- Types: full replication, partial replication, master-slave, peer-to-peer



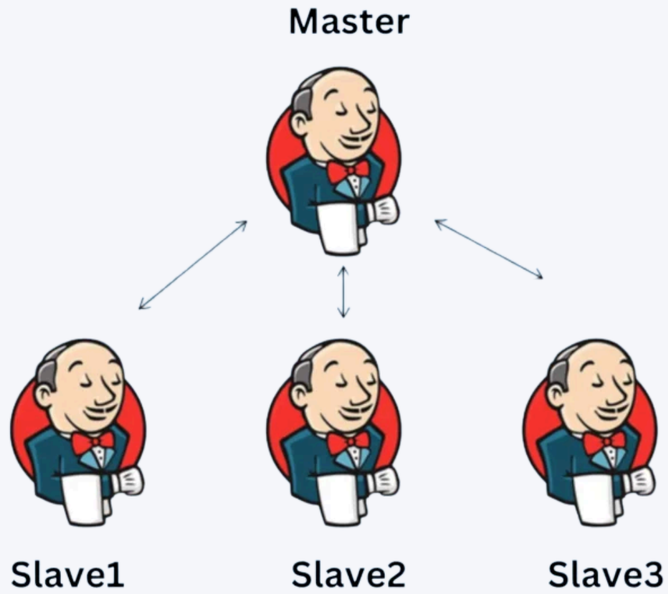
Distributed Computing Principles

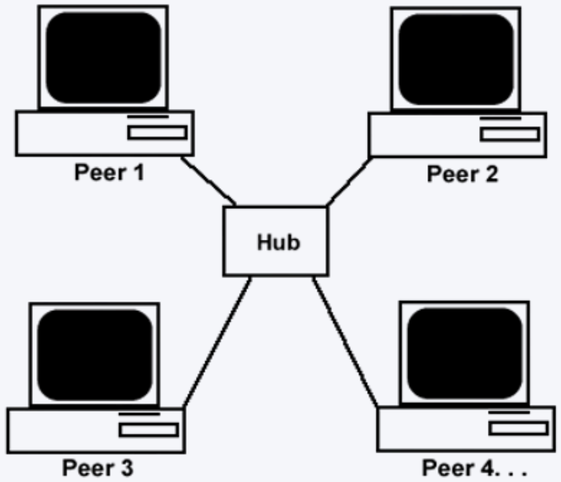
Three Main Architecture Patterns:

1. Master-Worker (Master-Slave)
- Central coordinator assigns tasks to workers
- Pros: Simple management, centralized control
- Cons: Single point of failure at master
- Examples: Hadoop MapReduce, traditional RDBMS replication

2. Peer-to-Peer:
- All nodes have equal roles and responsibilities
- Pros: No single point of failure, high resilience
- Cons: Complex coordination, eventual consistency challenges
- Examples: BitTorrent, blockchain networks, Cassandra

3. Hybrid Approaches:
- Mixing centralized and decentralized elements
- Examples: Kafka (distributed but with ZooKeeper coordination)





Data Preprocessing at Scale

Distributed Computing Principles

The Data Preprocessing Challenge

The 80/20 Rule of Data Science

- 80% of time spent on data preparation
- 20% of time spent on actual analysis and modeling

What Makes Preprocessing at **Scale** Different?

- Can't load all data into memory
- Can't manually inspect all records
- Performance bottlenecks are amplified
- New failure modes emerge
- Each operation has cost implications

Distributed Computing Principles

The Data Science Pipeline at Scale

أكاديمية طويق
Tuwaiq Academy

Traditional (Small Data) Pipeline:

```
Raw Data → Clean → Transform → Model → Deploy
```

Big Data Reality:

```
Raw Data → Sample → Explore → Design Pipeline →
Distributed Preprocessing → Validate → Model →
Validate Again → Optimize → Deploy → Monitor
```

Why can't we simply apply the same preprocessing techniques used for small data to big data scenarios?

Distributed Computing Principles

Preprocessing Challenges That Scale With Data Volume

Challenge	Small Data Approach	Big Data Reality
Missing values	Manual inspection, simple imputation	Statistical imputation, prediction models
Outliers	Visual detection, manual removal	Automated detection algorithms, robust methods
Feature engineering	Iterative testing of ideas	Automated feature generation, feature stores
Inconsistent formats	Manual standardization	Automated parsing, distributed ETL
Duplicate records	Direct comparison	Probabilistic matching, LSH
High cardinality categoricals	One-hot encoding	Target encoding, embeddings

Distributed Computing Principles

Data Validation at Scale: Ensuring Quality Input

Data Validation Strategies:

Schema Validation

- Ensuring data adheres to expected structure
- Handling schema drift and evolution
- Tools: Great Expectations, Deequ, tf.data validation

Statistical Profiling

- Automatically analyzing distributions and patterns
- Identifying potential issues before processing
- Example: Column statistics, correlation analysis

Anomaly Detection

- Finding unusual patterns or outliers
- Prevents corrupted data from affecting downstream analysis
- Methods: Statistical tests, clustering, isolation forests